

```

"""
Pump.fun Trading Bot - Optimierte Version
- Strenge Kauffilter für bessere Win-Rate
- Partial Take-Profit Strategie
- Alle Benachrichtigungen in Telegram
"""

import asyncio
import json
import time
import logging
import os
import aiohttp

# Config aus Umgebungsvariablen (Railway)
TELEGRAM_TOKEN = os.environ.get("TELEGRAM_TOKEN")
TELEGRAM_CHAT_ID = os.environ.get("TELEGRAM_CHAT_ID")
RPC_URL = os.environ.get("RPC_URL")
PRIVATE_KEY = os.environ.get("PRIVATE_KEY")
TRADE_AMOUNT_SOL = float(os.environ.get("TRADE_AMOUNT_SOL", "0.1"))
TAKE_PROFIT_1 = 0.50 # 50% → erste Hälfte verkaufen
TAKE_PROFIT_2 = 1.00 # 100% → zweite Hälfte verkaufen
STOP_LOSS = 0.40 # -40% → alles verkaufen
MAX_CONCURRENT_TRADES = int(os.environ.get("MAX_CONCURRENT_TRADES", "3"))
MAX_MARKETCAP_USD = 40000 # Max MarketCap in USD

logging.basicConfig(level=logging.INFO, format='%(asctime)s | %(levelname)s | %(m
log = logging.getLogger(__name__)

# Aktive Trades
active_trades = {}
bot_running = False
total_pnl = 0.0
total_trades = 0
winning_trades = 0

# Manuelle Verkauf-Anfragen
manual_sell_requests = set()

async def send_telegram(message: str, reply_markup=None):
    """Nachricht an Telegram senden"""
    if not TELEGRAM_TOKEN or not TELEGRAM_CHAT_ID:
        return

```

[illegible]

```

        data = await resp.json()
        accounts = data.get("result", {}).get("value", [])
        return len(accounts)

except:
    pass
return 0

async def get_token_price(token_mint: str) -> float:
    """Token Preis holen"""
    try:
        async with aiohttp.ClientSession() as session:
            url = f"https://price.jup.ag/v6/price?ids={token_mint}&vsToken=Sol1111"
            async with session.get(url, timeout=aiohttp.ClientTimeout(total=5)) as resp:
                if resp.status == 200:
                    data = await resp.json()
                    price = data.get("data", {}).get(token_mint, {}).get("price", 0)
                    return float(price)
    except:
        pass
    return 0.0

async def analyze_token(data: dict) -> tuple[bool, str]:
    """
    Token analysieren - gibt (kaufen, grund) zurück

    GUTE SIGNALE (kaufen):
    - Mehrere bekannte Wallets kaufen gleichzeitig
    - Hoher initialer Kauf (>2 SOL)
    - Token hat Social Media Links
    - MarketCap unter 40.000$

    SCHLECHTE SIGNALE (nicht kaufen):
    - Nur ein einziger großer Käufer
    - Kein Social Media
    - Sehr niedriger initialer Kauf (<1 SOL)
    - MarketCap über 40.000$
    """
    name = data.get("name", "")
    symbol = data.get("symbol", "")
    initial_buy = data.get("initialBuy", 0)
    market_cap_sol = data.get("marketCapSol", 0)
    twitter = data.get("twitter", "")
    telegram = data.get("telegram", "")
    website = data.get("website", "")
    top_10_holders = data.get("top10HoldersPercent", 100)

```

```

# SOL Preis für MarketCap Berechnung
sol_price = await get_sol_price_usd()
market_cap_usd = market_cap_sol * sol_price

# ❌ FILTER 1: MarketCap zu hoch
if market_cap_usd > MAX_MARKETCAP_USD:
    return False, f"MarketCap zu hoch: ${market_cap_usd:.0f}"

# ❌ FILTER 2: Initialer Kauf zu niedrig (schwaches Interesse)
if initial_buy < 1.0:
    return False, f"Initialer Kauf zu niedrig: {initial_buy:.2f} SOL"

# ✅ GUTES SIGNAL: Hoher initialer Kauf
score = 0
reasons = []

if initial_buy >= 3.0:
    score += 3
    reasons.append(f"✅ Starker Kauf: {initial_buy:.1f} SOL")
elif initial_buy >= 2.0:
    score += 2
    reasons.append(f"✅ Guter Kauf: {initial_buy:.1f} SOL")
else:
    score += 1
    reasons.append(f"⚠️ Kauf: {initial_buy:.1f} SOL")

# ✅ GUTES SIGNAL: Niedriger MarketCap = mehr Potenzial
if market_cap_usd < 10000:
    score += 2
    reasons.append(f"✅ Früher Einstieg: ${market_cap_usd:.0f}")
elif market_cap_usd < 25000:
    score += 1
    reasons.append(f"✅ MarketCap: ${market_cap_usd:.0f}")

# Mindest-Score für Kauf
if score >= 2:
    return True, "\n".join(reasons)
else:
    return False, f"Score zu niedrig: {score}/4"

async def buy_token(token_mint: str, amount_sol: float) -> bool:
    """Token kaufen"""
    try:

```

```

    async with aiohttp.ClientSession() as session:
        payload = {
            "action": "buy",
            "mint": token_mint,
            "amount": amount_sol,
            "denominatedInSol": "true",
            "slippage": 15,
            "priorityFee": 0.001,
            "pool": "pump"
        }
        async with session.post(
            "https://pumpportal.fun/api/trade-local",
            json=payload,
            timeout=aiohttp.ClientTimeout(total=10)
        ) as resp:
            return resp.status == 200
except Exception as e:
    log.error(f"Kauf Fehler: {e}")
    return False

async def sell_token(token_mint: str, percent: str = "100%") -> bool:
    """Token verkaufen (percent = "50%" oder "100%")"""
    try:
        async with aiohttp.ClientSession() as session:
            payload = {
                "action": "sell",
                "mint": token_mint,
                "amount": percent,
                "denominatedInSol": "false",
                "slippage": 20,
                "priorityFee": 0.001,
                "pool": "pump"
            }
            async with session.post(
                "https://pumpportal.fun/api/trade-local",
                json=payload,
                timeout=aiohttp.ClientTimeout(total=10)
            ) as resp:
                return resp.status == 200
    except Exception as e:
        log.error(f"Verkauf Fehler: {e}")
        return False

async def monitor_positions():
    """Positionen überwachen mit Partial Take-Profit"""

```

```

global total_pnl, total_trades, winning_trades

while True:
    try:
        for token_mint, trade in list(active_trades.items()):
            if not bot_running:
                break

            # Manueller Verkauf angefordert?
            if token_mint in manual_sell_requests:
                success = await sell_token(token_mint, "100%")
                if success:
                    manual_sell_requests.discard(token_mint)
                    del active_trades[token_mint]
                    await send_telegram(
                        f"✅ <b>Manuell verkauft!</b>\n"
                        f"Token: {trade['name']}"
                    )
                    continue

                current_price = await get_token_price(token_mint)
                if current_price == 0:
                    continue

                entry_price = trade["entry_price"]
                if entry_price == 0:
                    continue

                pnl_percent = ((current_price - entry_price) / entry_price) * 100
                pnl_sol = TRADE_AMOUNT_SOL * (pnl_percent / 100)

                # TAKE-PROFIT STUFE 1: +50% → erste Hälfte verkaufen
                if pnl_percent >= TAKE_PROFIT_1 * 100 and not trade.get("tp1_done"):
                    success = await sell_token(token_mint, "50%")
                    if success:
                        active_trades[token_mint]["tp1_done"] = True
                        await send_telegram(
                            f"📊 <b>TAKE-PROFIT 1 (50%)</b>\n"
                            f"Token: {trade['name']}\n"
                            f"Gewinn: +{pnl_percent:.1f}%\n"
                            f"Hälfte gesichert: +{pnl_sol/2:.3f} SOL\n\n"
                            f"Rest läuft weiter bis +100% oder Stop-Loss\n"
                            f"Tippe /sell_{token_mint[:8]} für manuellen Verkauf"
                        )
                        reply_markup={
                            "inline_keyboard": [[
                                {"text": "🔴 Alles verkaufen", "callback_data":

```

```

    }
)

# TAKE-PROFIT STUFE 2: +100% → Rest verkaufen
elif pnl_percent >= TAKE_PROFIT_2 * 100 and trade.get("tp1_done"):
    success = await sell_token(token_mint, "100%")
    if success:
        total_trades += 1
        winning_trades += 1
        total_pnl += pnl_sol
        del active_trades[token_mint]
        await send_telegram(
            f"🎯 <b>TAKE-PROFIT 2 (+100%)</b>\n"
            f"Token: {trade['name']}\n"
            f"Gesamt Gewinn: +{pnl_percent:.1f}%\n"
            f"Gesamt PnL: {total_pnl:+.3f} SOL"
        )

# STOP-LOSS: -40%
elif pnl_percent <= -(STOP_LOSS * 100):
    success = await sell_token(token_mint, "100%")
    if success:
        total_trades += 1
        total_pnl += pnl_sol
        del active_trades[token_mint]
        await send_telegram(
            f"🛑 <b>STOP-LOSS (-40%)</b>\n"
            f"Token: {trade['name']}\n"
            f"Verlust: {pnl_percent:.1f}% ({pnl_sol:.3f} SOL)\n"
            f"Gesamt PnL: {total_pnl:+.3f} SOL"
        )

except Exception as e:
    log.error(f"Monitor Fehler: {e}")

await asyncio.sleep(5)

async def watch_new_tokens():
    """Neue Pump.fun Tokens überwachen"""
    while True:
        if not bot_running:
            await asyncio.sleep(5)
            continue

        try:
            async with aiohttp.ClientSession() as session:

```

```

        async with session.ws_connect(
            "wss://pumpportal.fun/api/data",
            timeout=aiohttp.ClientTimeout(total=30)
        ) as ws:
            await ws.send_json({"method": "subscribeNewToken"})
            log.info("✅ Verbunden mit Pump.fun")

            async for msg in ws:
                if not bot_running:
                    break
                if msg.type == aiohttp.WSMsgType.TEXT:
                    data = json.loads(msg.data)
                    await handle_new_token(data)

    except Exception as e:
        log.error(f"WebSocket Fehler: {e}")
        await asyncio.sleep(10)

async def handle_new_token(data: dict):
    """Neuen Token analysieren und ggf. kaufen"""
    token_mint = data.get("mint", "")
    if not token_mint:
        return

    if len(active_trades) >= MAX_CONCURRENT_TRADES:
        return

    if token_mint in active_trades:
        return

    name = data.get("name", "Unbekannt")
    symbol = data.get("symbol", "?")

    # Token analysieren
    should_buy, reason = await analyze_token(data)

    if not should_buy:
        log.info(f"⏩ Übersprungen: {name} | {reason}")
        return

    # Kaufen!
    await send_telegram(
        f"🆕 <b>Token gefunden!</b>\n"
        f"Name: {name} (${symbol})\n"
        f"{reason}\n\n"
        f"Kaufe {TRADE_AMOUNT_SOL} SOL..."
    )

```

```

)

entry_price = await get_token_price(token_mint)
success = await buy_token(token_mint, TRADE_AMOUNT_SOL)

if success:
    active_trades[token_mint] = {
        "entry_price": entry_price,
        "amount": TRADE_AMOUNT_SOL,
        "entry_time": time.time(),
        "name": name,
        "symbol": symbol,
        "tp1_done": False,
        "tp2_done": False
    }
    await send_telegram(
        f"✅ <b>Kauf erfolgreich!</b>\n"
        f"Token: {name} ({symbol})\n"
        f"Einsatz: {TRADE_AMOUNT_SOL} SOL\n"
        f"TP1: +50% (Hälfte)\n"
        f"TP2: +100% (Rest)\n"
        f"SL: -40% (Alles)"
    )
else:
    await send_telegram(f"❌ Kauf fehlgeschlagen: {name}")

async def handle_telegram_commands():
    """Telegram Befehle verarbeiten"""
    global bot_running
    offset = 0

    await send_telegram(
        f"🚀 <b>Trading Bot gestartet!</b>\n\n"
        "Befehle:\n"
        "/start - Bot starten\n"
        "/stop - Bot stoppen\n"
        "/status - Aktueller Status\n"
        "/trades - Aktive Trades\n"
        "/pnl - Gewinn/Verlust\n"
        "/sellall - Alle Positionen verkaufen"
    )

while True:
    try:
        async with aiohttp.ClientSession() as session:
            url = f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/getUpdates?o

```

```

async with session.get(url, timeout=aiohttp.ClientTimeout(total=3
    if resp.status == 200:
        data = await resp.json()
        for update in data.get("result", []):
            offset = update["update_id"] + 1

# Button Callbacks
callback = update.get("callback_query", {})
if callback:
    cb_data = callback.get("data", "")
    if cb_data.startswith("sell_"):
        token_mint = cb_data.replace("sell_", "")
        manual_sell_requests.add(token_mint)
        await send_telegram("🔴 Manueller Verkauf wir
        continue

message = update.get("message", {})
text = message.get("text", "")

if text == "/start":
    bot_running = True
    await send_telegram(
        "▶ <b>Bot läuft!</b>\n"
        f"Einsatz: {TRADE_AMOUNT_SOL} SOL\n"
        f"TP1: +50% (50% verkaufen)\n"
        f"TP2: +100% (Rest verkaufen)\n"
        f"Stop-Loss: -40%\n"
        f"Max MarketCap: ${MAX_MARKETCAP_USD:,}\n"
        f"Max. Trades: {MAX_CONCURRENT_TRADES}"
    )

elif text == "/stop":
    bot_running = False
    await send_telegram("⏸ <b>Bot pausiert.</b> Akti

elif text == "/status":
    status = "▶ Läuft" if bot_running else "⏸ Pausi
    win_rate = (winning_trades / total_trades * 100)
    await send_telegram(
        f"📊 <b>Status</b>\n"
        f"Bot: {status}\n"
        f"Aktive Trades: {len(active_trades)}\n"
        f"Gesamt Trades: {total_trades}\n"
        f"Win Rate: {win_rate:.1f}%\n"
        f"Gesamt PnL: {total_pnl:+.3f} SOL"
    )

```

```

elif text == "/trades":
    if not active_trades:
        await send_telegram("🚫 Keine aktiven Trades.")
    else:
        msg = "📈 <b>Aktive Trades:</b>\n\n"
        for mint, trade in active_trades.items():
            duration = (time.time() - trade["entry_time"])
            current = await get_token_price(mint)
            if trade["entry_price"] > 0 and current > trade["entry_price"]:
                pnl = ((current - trade["entry_price"]) / trade["entry_price"]) * 100
                tp1 = "✅" if trade.get("tp1_done") else "❌"
                msg += f"• {trade['name']} (${trade['entry_price']} → {current}) | PnL: {pnl:.1f}% | TP1: {trade['tp1']} {tp1}\n"
            else:
                msg += f"• {trade['name']} (${trade['entry_price']} → {current}) | PnL: {pnl:.1f}% | TP1: {trade['tp1']} ❌\n"
        await send_telegram(msg)

elif text == "/pnl":
    win_rate = (winning_trades / total_trades * 100)
    await send_telegram(
        f"💰 <b>Gewinn/Verlust</b>\n\n"
        f"Gesamt PnL: {total_pnl:+.3f} SOL\n\n"
        f"Trades: {total_trades}\n\n"
        f"Gewonnen: {winning_trades}\n\n"
        f"Win Rate: {win_rate:.1f}%"
    )

elif text == "/sellall":
    if not active_trades:
        await send_telegram("Keine aktiven Trades.")
    else:
        for mint in list(active_trades.keys()):
            manual_sell_requests.add(mint)
        await send_telegram(f"🔴 Verkaufe alle {len(active_trades)} Trades.")

except Exception as e:
    log.error(f"Telegram Command Fehler: {e}")
    await asyncio.sleep(5)

async def main():
    log.info("🚀 Optimierter Trading Bot startet...")
    await asyncio.gather(
        handle_telegram_commands(),
        watch_new_tokens(),
        monitor_positions(),
    )

```

```
if __name__ == "__main__":  
    asyncio.run(main())
```